

AI-Native Kernel

Next Generation Operating System Kernel for AI Era — Rust Implementation Plan

Version: 1.0 (Detailed Plan)

Language: Rust (Edition 2021)

Status: Ready for Prototype

1. ภาพรวมสถาปัตยกรรมระดับต่ำ (Low-Level Architecture Architecture)

การพัฒนา AI-Native Kernel ใน Phase 1 จะใช้แนวทาง **Hybrid-Companion Model** โดยทำงานร่วมกับ Linux Kernel ผ่านระบบ **eBPF** และ **LSM** เพื่อดักจับพฤติกรรมในระดับริงศูนย์ (Kernel Space) และส่งต่อการควบคุมมายังระบบหลักที่พัฒนาขึ้นด้วยภาษา Rust ซึ่งรันอยู่บนโครงสร้างประสิทธิภาพสูงเพื่อการขับเคลื่อนด้วย AI

ทำไมต้องใช้ Rust?

- Memory Safety Without GC:** ป้องกันข้อผิดพลาดเกี่ยวกับหน่วยความจำ (เช่น Use-After-Free) ซึ่งสำคัญมากในการจัดการ Context Graph ขนาดใหญ่
- Async/Await (Tokio):** รองรับการทำ Concurrency ของ AI Agents และ I/O Operations ได้พร้อมกันหลายหมื่นตัวโดยไม่เกิด Overhead
- FFI & C Interoperability:** สามารถเชื่อมต่อกับ Linux System Calls และโครงสร้าง C ของ eBPF ได้อย่างไร้รอยต่อ

2. แผนการพัฒนารายโมดูล (Detailed Module Implementation)

Module 1: Agent Scheduler (ระบบจัดตารางเวลาและควบคุม Agent)

ทำหน้าที่ควบคุมวงจรชีวิต (Lifecycle) ระดับความสำคัญ (Priority) และการแยกส่วน (Isolation) ของ Agent แต่ละตัว โดยใช้ระบบควบคุมผ่าน **tokio::runtime** ในการสร้าง Light-weight Green Threads สำหรับขบวนการของ Agent

ตัวอย่างโครงสร้างข้อมูล (Rust Struct):

```
pub enum AgentState {
    Ready,
    Running,
    WaitingForHumanApproval,
    WaitingForContext,
    Suspended,
    Terminated,
}

pub struct AgentControlBlock {
    pub agent_id: u64,
    pub name: String,
    pub priority: u8,
    pub state: AgentState,
    pub capabilities: Vec<CapabilityToken>,
    pub context_id: u64,
}
```

Module 2: Context Memory Manager (ระบบจัดการหน่วยความจำบริบท)

ออกแบบโดยอาศัยหลักการ **Context Paging Memory** เพื่อสลับเปลี่ยนข้อมูลบริบทระหว่างเลเยอร์ความเร็วสูง (VRAM) และเลเยอร์ถาวร (NVMe)

- **Hot Context:** เก็บไว้ใน GPU VRAM / NPU Cache โดยใช้รูปแบบ **Tensor** เพื่อให้พร้อมสำหรับการ Inference ทันที
- **Warm Context:** พักไว้ใน System RAM ในรูปแบบของโครงสร้าง **Vector (f32)**
- **Cold Context:** ทำการ Serialization ลงในดิสก์ผ่านระบบไฟล์ความเร็วสูงโดยใช้ **RocksDB** หรือคลังข้อมูลเวกเตอร์แบบฝัง (Embedded Vector Store)

Module 3: Compute Scheduler & Decision Matrix

ทำหน้าที่วิเคราะห์เชิงสถิติเพื่อคัดเลือกหน่วยประมวลผลที่ประหยัดพลังงานที่สุดและได้ผลลัพธ์เร็วที่สุด โดยใช้สมการคำนวณ Cost Function ดังนี้:

$$Cost = (w_1 \times Latency) + (w_2 \times Power) + (w_3 \times FinancialCost)$$

ประเภท Task / workload	CPU (Tokio Worker)	GPU (via TensorRT)	NPU (via ONNX/ Ollama)	Cloud Execution
Kernel Logic / LSM Check	✓ (High Priority)	—	—	—
Small LLM (Under 7B)	—	—	✓ (Optimal Latency)	—
Large LLM (Embedding/ Reasoning)	—	✓ (Batch Run)	—	✓ (If Budget Allows)
Semantic Vector Indexing	—	✓ (Parallel)	✓ (Streaming)	—

Module 4: Capability & Security Manager (ระบบรักษาความปลอดภัยแบบ Zero-Trust)

ห้ามไม่ให้ Agent เข้าถึง Linux API โดยตรง โดยทุก System Calls จะถูกดักจับที่ระดับลินุกซ์เคอร์เนลผ่าน **LSM Hook** แล้วส่งผ่าน IPC มาตรวจสอบที่ระบบนโยบายความปลอดภัย (Policy Engine) ของเรา

```
// โครงสร้างการตรวจสอบสิทธิ์ระดับรากฐาน
pub struct CapabilityToken {
    pub token_id: [u8; 32],
    pub allowed_action: String,
    pub resource_mask: String,
    pub max_cost_limit: u64,
}
```

3. ลำดับเหตุการณ์และขั้นตอนการรันโปรแกรม (Execution Pipeline)

เมื่อผู้ใช้ส่งความต้องการ (Intent/Goal) เข้ามาในระบบ วงจรการทำงานภายใน AI-Native Kernel จะถูกส่งต่อผ่านบัสสัญญาณตามขั้นตอนดังต่อไปนี้:

- 1. **Intent Bus Capture:** สัญญาณเป้าหมายถูกจับคู่และแปลงให้กลายเป็นโครงสร้าง Intent Event ข้อมูลจะถูกประมวลผลผ่านตัวรับสัญญาณระบบ (System Event Loop)
- 2. **Agent Spawning:** Agent Scheduler ทำการสร้างสแต็กและหน่วยความจำจำลองของ Agent พร้อมแนบ Capability Token ประจำตัว
- 3. **Context Attachment:** Context Memory Manager โหลดไฟล์และหน่วยความจำเวกเตอร์ที่เกี่ยวข้องทั้งหมดขึ้นมาจัดเตรียมในสถานะ Hot Context
- 4. **Tool Execution & Sandboxing:** หาก Agent ร้องขอการเรียกใช้เครื่องมือ ระบบ Tool Broker จะทำการครอบคำสั่งและรันภายใน gRPC Sandbox พร้อมทั้งบันทึกลงใน Audit Logger แบบถาวรที่ไม่สามารถลบได้

4. แผนงานการพัฒนา 4 เฟส (Development Roadmap)

เฟสการทำงาน (Phase)	เป้าหมายหลัก (Milestone Goals)	เทคโนโลยีที่ใช้งาน (Stack Selected)
Phase 1: Linux Prototype	สร้างตัวแทนระบบ (Kernel Companion) ดักจับ Syscall และสร้างระบบจำลอง Agent พื้นฐานบนระบบปฏิบัติการเดิม	Rust, eBPF (Aya), LSM, SQLite, gRPC
Phase 2: AI Runtime	ระบบสลับหน่วยความจำ Context Paging สำเร็จ, และ Compute Scheduler สามารถขับโหลดงานลง GPU/NPU ได้อัตโนมัติ	Tokio, llama.cpp, TensorRT, RocksDB
Phase 3: Semantic OS	เปลี่ยนผ่านโครงสร้างระบบไฟล์เดิมให้กลายเป็น Semantic File System ค้นหาไฟล์ด้วยความหมายแบบ Native	Qdrant Embedded, Neo4j, Rust VFS
Phase 4: Distributed AI OS	ขยายขีดความสามารถให้รองรับการทำงานข้ามเครื่องผ่านระบบเครือข่ายประมวลผลร่วม (Cluster Compute Sharing)	NATS Messaging, Libp2p, Distributed KV